

```

import nltk
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score
from textblob import TextBlob
import pandas as pd

# Download necessary NLTK data for the first time
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)

def process_text(text):
    print("\n--- Text Processing Steps ---")
    print(f"1. Original Text:\n{text}\n")

    # 2. Text cleaning and normalization
    cleaned_text = re.sub(r'^[a-zA-Z\s]', '', text).lower()
    print(f"2. Cleaned and Normalized Text:\n{cleaned_text}\n")

    # 3. Tokenization
    tokens = word_tokenize(cleaned_text)
    print(f"3. Tokenized Text:\n{tokens}\n")

    # 4. Remove stop words
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [word for word in tokens if word not in stop_words]
    print(f"4. After Stop Words Removal:\n{filtered_tokens}\n")

    # 5. Stemming and Lemmatization
    stemmer = PorterStemmer()
    lemmatizer = WordNetLemmatizer()

    stemmed_tokens = [stemmer.stem(word) for word in filtered_tokens]
    print(f"5a. Stemmed Text:\n{stemmed_tokens}\n")

    lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]
    print(f"5b. Lemmatized Text:\n{lemmatized_tokens}\n")

    # Return processed text as string
    return " ".join(lemmatized_tokens)

def classify_sentiment(text):
    # 7. Classify the processed text (using TextBlob for simplicity)
    analysis = TextBlob(text)
    if analysis.sentiment.polarity >= 0:
        return "Positive"
    else:
        return "Negative"

def evaluate_system():
    # Dummy dataset for evaluation (To generate confusion matrix and metrics)
    print("\n" + "="*40)
    print("--- System Evaluation on Test Dataset ---")
    print("="*40)

```

```

data = [
    ("I love this amazing product, it is absolutely wonderful!", "Positive"),
    ("This is the worst experience I have ever had. Terrible.", "Negative"),
    ("I am very happy with the results, great job.", "Positive"),
    ("I hate this, it is so bad and useless.", "Negative"),
    ("The service was okay, but I expected better.", "Negative"),
    ("Fantastic work, highly recommended!", "Positive")
]

actual_labels = []
predicted_labels = []

print("\nEvaluating test samples...\n")
for text, actual in data:
    # Simplify processing for evaluation output to keep it clean
    cleaned = re.sub(r'^a-zA-Z\s', '', text).lower()
    predicted = classify_sentiment(cleaned)

    # Mapping to binary labels: Positive -> 1, Negative -> 0
    actual_labels.append(1 if actual == "Positive" else 0)
    predicted_labels.append(1 if predicted == "Positive" else 0)

# 8. Confusion Matrix
cm = confusion_matrix(actual_labels, predicted_labels)
print("8. Confusion Matrix (0: Negative, 1: Positive):")
print(pd.DataFrame(cm, index=['Actual Neg', 'Actual Pos'], columns=['Pred Neg', 'Pred Pos']))
print("\n")

# 9. Precision, Recall, F1-Score
precision = precision_score(actual_labels, predicted_labels, zero_division=0)
recall = recall_score(actual_labels, predicted_labels, zero_division=0)
f1 = f1_score(actual_labels, predicted_labels, zero_division=0)

# 10. Display Final Evaluation Results
print("10. Final Evaluation Results:")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-Score: {f1:.2f}")

if __name__ == "__main__":
    print("=== NLP Text Evaluation System ===\n")

    # 1. Read input text from the user
    user_input = input("Enter a paragraph to evaluate: ")

    if user_input.strip():
        # Process user input and display step 1 through 6
        processed_input = process_text(user_input)

        # 7. Classify the processed text as Positive or Negative
        sentiment = classify_sentiment(processed_input)
        print(f"7. Classification:\n => The input text is classified as: {sentiment}\n")

    # Execute steps 8, 9, 10
    evaluate_system()

```

=== NLP Text Evaluation System ===

Enter a paragraph to evaluate: I love this amazing product, it is absolutely wonderful!

--- Text Processing Steps ---

1. Original Text:

I love this amazing product, it is absolutely wonderful!

2. Cleaned and Normalized Text:

i love this amazing product it is absolutely wonderful

3. Tokenized Text:

['i', 'love', 'this', 'amazing', 'product', 'it', 'is', 'absolutely', 'wonderful']

4. After Stop Words Removal:

['love', 'amazing', 'product', 'absolutely', 'wonderful']

5a. Stemmed Text:

['love', 'amaz', 'product', 'absolut', 'wonder']

5b. Lemmatized Text:

['love', 'amazing', 'product', 'absolutely', 'wonderful']

7. Classification:

=> The input text is classified as: Positive

=====

--- System Evaluation on Test Dataset ---

=====

Evaluating test samples...

8. Confusion Matrix (0: Negative, 1: Positive):

	Pred Neg	Pred Pos
Actual Neg	2	1
Actual Pos	0	3

10. Final Evaluation Results:

Precision: 0.75

Recall: 1.00

F1-Score: 0.86